

The propositions package*

Cian Dorr (with help from Claude)
ciandorr@gmail.com

2026/02/13

Abstract

The `propositions` package provides a key-value driven system for labelling propositions, theses, and premises in academic papers. Items may be given names like ‘(P)’ or ‘Physicalism’, or auto-numbered using different counters; all carry robust cross-references with configurable formatting. The package integrates with `amsmath`, `hyperref`, and `cleveref`.

Contents

1	Introduction	2
2	History	2
3	Basic usage	2
4	Advanced usage	3
5	Basic environments and commands	4
6	Keys	5
6.1	Item-level keys	5
6.2	Keys for <code>prop</code> environments	7
6.3	Keys for <code>prop</code> and <code>inlineprop</code> environments	9
6.4	Global keys	10
7	Defining and modifying styles	10
8	Built-in styles	12
9	Cross-referencing commands	15
9.1	How cross-referencing works: <code>\propapply</code>	17
10	Compatibility	17
11	Known issues	18

*This document describes version v0.2, dated 2026/02/13.

1 Introduction

In some academic disciplines (such as philosophy), it is common to have displayed propositions (examples, theses, premises, . . .) with various kinds of labels. A thesis might be referred to as ‘(P)’ or ‘Physicalism’; the premises of an argument might be numbered as ‘P1’, ‘P2’, ‘P3’, . . . ; or examples might be numbered consecutively over the course of a whole article. Standard L^AT_EX environments like `enumerate` can handle some of these cases, but cross-referencing is awkward: `\ref` produces a bare number or letter, and the author must manually add parentheses or other formatting at every point of reference. The standard `description` environment does not allow cross-referencing at all.

The `propositions` package solves this by attaching formatting information to each label. A short item like `\item[P]` (inside `prop`) is displayed as “(P)” and `\ref` automatically produces “(P)” as well—complete with parentheses, hyperlinks, and `cleveref` support. The full key-value interface supports named items, numbered items, custom counters, glosses, shorthands, and per-item format overrides.

The `\ptag` command (which requires `amsmath`) extends this to displayed math environments: an equation can be tagged with a proposition label instead of (or using) its equation number.

2 History

I wrote the ancestor to this package in the 90s while finishing my Ph.D. thesis, but never documented it or shared it with the world. This new version is a thorough re-implementation in L^AT_EX3, written in 2026 with extensive help from Claude Code. I hope others will find it as useful as I have.

3 Basic usage

Load the package with `\usepackage{propositions}` or `\usepackage[options]{propositions}` (see ?? below for valid package options).

The `prop` environment generates a list of propositions, each introduced by `\item`. `\item` with an optional argument gives a `description`-like label:

<pre>\begin{prop} \item[Physicalism] Everything is physical. \label{phys} \item[Idealism] Everything is mental. \label{ideal} \end{prop}</pre>	<pre>Physicalism Everything is physi- cal. Idealism Everything is mental.</pre>
--	---

Unlike the standard `description` environment, one can refer back to these propositions using the standard `\ref` command:

<pre>\ref{phys} is more plausible than \ref{ideal}.</pre>	<pre>Physicalism is more plausible than Idealism.</pre>
---	---

With no optional argument, `\item` will by default generate numbered items similar to `enumerate`, but with numbering that persists across the document:

<pre> \begin{prop} \item Every atom is physical. \label{atoms} \end{prop} \ref{phys} follows from the conjunction of \ref{atoms} and \begin{prop} \item Everything is an atom. \label{atomism} \end{prop} </pre>	(1) Every atom is physical. Physicalism follows from the conjunction of (1) and (2) Everything is an atom.
--	--

As with `enumerate`, the counter and formatting depend on the nesting level:

<pre> \begin{prop} \item \label{dual} \begin{prop} \item Some things are physical. \label{some} \item Some things are not physical. \label{notall} \end{prop} \end{prop} </pre>	(3) a. Some things are physical. b. Some things are not physical. Without (3a), (3) would be consistent with Idealism.
---	---

Without `\ref{some}`, `\ref{dual}` would be consistent with `\ref{ideal}`.

4 Advanced usage

The format of the proposition labels, and of subsequent references, are both configurable using a key=value syntax (see [section 6](#) for the possible keys):

<pre> \begin{prop} \item[No Overlap, align=flush, display format=\textsc{#1}, ref format=\textit{#1}] Nothing mental is physical. \label{incomp} \end{prop} </pre>	No OVERLAP Nothing mental is physical. Is <i>No Overlap</i> consistent with Physicalism?
--	---

Is `\ref{incomp}` consistent with `\ref{phys}`?

The `prop` environment can also take an optional argument with a list of keys:

<pre> \begin{prop}[leftmargin=2em, rightmargin=2em, display format=#1] \item Every mental thing is physical. \end{prop} </pre>	[4] Every mental thing is physical.
--	--

Preset styles can be declared and used in place of a set of key-value pairs:

<pre> \begin{prop} \item[Nihilism, style=thesis] There is nothing. \label{nihilism} \end{prop} Does \ref{nihilism} imply \ref{phys}, \ref{dual}, or both? Discuss. </pre>	NIHILISM There is nothing. Does NIHILISM imply Physicalism, (3), or both? Discuss.
---	---

Shortcut commands—e.g., `\titem[⟨keys⟩]` for `\item[style=thesis, ⟨keys⟩]`—can also be defined. The package loads with a range of predefined styles (listed in section 8).

When the package is loaded with `\usepackage[equations]{propositions}`, a first-level `\item` within `prop` will use the counter as equations. (This looks better with the `leqno` option to `\documentclass`.)

<pre> \begin{equation} \exists x (\text{Mental}(x) \wedge \text{Physical}(x)) \end{equation} \begin{prop} \item There is overlap between the mental and the physical. \end{prop} </pre>	(5) $\exists x(\text{Mental}(x) \wedge \text{Physical}(x))$ (6) There is overlap between the mental and the physical.
---	--

The `\ptag` command (requires `amsmath`) is a replacement for the standard `\tag` command that behaves just like an `\item` in a `prop` environment.

<pre> \begin{equation} \ptag[Monism] \label{mon} \exists x \forall y (y = x) \end{equation} Is \ref{mon} compatible with \ref{dual}? </pre>	Monism $\exists x \forall y (y = x)$ Is Monism compatible with (3)?
---	---

5 Basic environments and commands

```

\begin{prop}[⟨keys⟩]
  ⟨environment content⟩
\end{prop}

```

Creates a displayed list of propositions. It is a standard L^AT_EX list, so by default its formatting will depend on the standard length parameters like `\itemsep` and `\topsep`, although these can be overridden by setting keys.

Within `prop`, `\item` creates the propositions (see below).

The optional `⟨keys⟩` argument can contain a list of keys, which will affect only the given environment (not any other `prop` environments that may be nested within it).

```

\begin{inlineprop}[⟨keys⟩]
  ⟨environment content⟩
\end{inlineprop}

```

Like `prop`, but does not create a list. Allows `\item` to be used outside list environments, e.g. for generating numbers at the beginning of paragraphs. Steps the `prop` counter and increments the nesting level. Accepts the same optional $\langle keys \rangle$ as `prop`.

Within `prop` and `inlineprop`, `\item` is redefined to act as the proposition-item command. Its optional argument is a comma-separated list of $\langle key \rangle = \langle value \rangle$ pairs (see section 6). A bare string without `=` is treated as a proposition name.

When used without an optional argument (or without setting `name`, `counter`, or `style`), the style is determined by the `nameless style` key (section 6). By default this is `numbered`, which dispatches by nesting depth to `levelone`, `leveltwo`, ... via `\proplevelchoice`.

`\ptag[ $\langle keys \rangle$ ]`

(Available only when `amsmath` is loaded.) Works inside displayed math environments like `equation` and `align`. Accepts the same keys as `\item` within `prop`, except that `align` has no effect (since positioning is controlled by the tag placement system).

Any display math environment (`equation`, `align`, etc.) used inside `prop` or `inlineprop` automatically increments the nesting level for its duration, so that `\ptag` inside such an environment behaves as if it were one level deeper.

`\propoptions{ $\langle keys \rangle$ }`

Sets default keys (see section 6), which take effect for all subsequent uses of `prop`, `inlineprop`, `\item`, and `\ptag` (within the current scope).

Further commands are described below, especially in section 7 (defining styles) and section 9 (cross-referencing).

6 Keys

6.1 Item-level keys

The keys listed in this subsection can be used in the following places:

- In the optional argument of `\item` or `\ptag`: the key applies to that item only.
- In the optional argument of `prop` or `inlineprop`: the key applies to every top-level `\item` within that environment, unless overridden by the `\item`'s own optional argument. (If another `prop` or `inlineprop` environment is used inside this one, the key will not apply to its `\items`.)
- In the argument of `\propoptions`: the key will apply to every subsequent `\item` or `\ptag` (in the current group), unless overridden by that `\item`'s optional argument, or that of its parent `prop` or `inlineprop` environment.
- In the optional argument of `\usepackage{propositions}`: equivalent to `\propoptions`.
- With `\SetPropStyle` or `\DeclareNumberedStyle`, to add the key to a given style.

`style=<style>` (no default)

A prop style, equivalent to a preset collection of keys: see section 7 for details.

`align=<value>` (no default)

How the label should be positioned. Possible values: `left` (standard left-aligned label, offset controlled by `\labelwidth` and `\labelsep`), `right` (right-aligned, like `enumerate`), `center` (centered within the label box), `flush` (aligned with left margin of item text), `nextline` (label on its own line), and `flush-nextline`. Has no effect inside `inlineprop` or `\ptag`. For left-margin alignment, use `labelindent=0em` instead.

`name=<text>` (no default)

The proposition's name. A bare string (without `=`) is equivalent to `name=<text>`. (To be precise: if the optional argument contains no `=` at all, the whole of it is the name; otherwise, it is read as a key list, and any entry of it without an `=` is the name, so a bare name can be combined with other keys, as in `\item[No Overlap, align=flush]`. In that case a name containing a comma must be braced—`\item[{Alpha, Beta}, align=flush]`—and a name containing an `=` needs the explicit `name={<text>}` form.)

`counter=<name>` (no default)

Counter to use. The counter is automatically stepped, and the item's `name` is set to `\the<name>` (though this can be overridden by `counter format` or `name`).

`counter format=<template>` (no default)

How to display the counter value. Use `#1` for the counter name, e.g. `counter format=\roman{#1}`. When both this key and `counter` are set, this is used instead of `\the<counter>` for generating the item's `name`.

`format=<template>` (no default)

Formatting applied to the `name`: use `#1` for the argument, e.g. `format=\textbf{(#1)}`. Shorthand for setting both `display format` and `ref format`.

`display format=<template>` (no default)

Format for displaying the name in the proposition's label. Does not affect cross-references.

`ref format=<template>` (no default)

Format for subsequent cross-references to this proposition. Does not affect the display.

`shorthand=<text>` (no default)

An abbreviation displayed after the name. If present, the shorthand becomes the reference text: `\ref` produces the shorthand (formatted with `ref format`) rather than the full name.

`shorthand format=<template>` (initially `\sim[#1]`)

Format for displaying the shorthand in the label.

`gloss=<text>` (no default)

A parenthetical gloss displayed after the name. Does not affect cross-references.

`gloss format`= $\langle template \rangle$ (initially $\sim(\#1)$)

Format for displaying the gloss in the label.

`label format`= $\langle template \rangle$ (no default)

Format applied to the *entire* assembled label, i.e. the name (after `display format`), shorthand, and gloss together. Use `#1` for the whole assembled text. For example, `label format=#1:` appends a colon after the complete label.

`ref`= $\langle text \rangle$ (no default)

Explicitly set the reference text, overriding what would be derived from `name`, `counter`, or `shorthand`.

`label`= $\langle label \rangle$ (no default)

Equivalent to a trailing `\label{\langle label \rangle}`.

`crefname`= $\langle type \rangle$ (no default)

When `cleveref` is loaded, assigns an arbitrary reference type to this proposition. For example, `crefname=lemma` causes `\cref` to use the names defined by `\crefname{lemma}{...}{...}` instead of the default (private) `prop` type. The $\langle type \rangle$ must be known to `cleveref`; new types can be declared with `\crefname`.

`reset`= $\langle boolean \rangle$ (initially `true`)

When `true` (the default), processing an `\item` resets the sub-level counter one nesting depth below the current level (e.g. `numpropii` at level 1). `reset=false` suppresses this behaviour, so that sub-item numbering continues from where it left off across consecutive parent items.

Four of the list-dimension keys documented in the next section—`labelwidth`, `labelsep`, `itemindent` and `labelindent`—may also be given to an individual `\item`, where they reposition the label of that item alone, overriding the value in force for the rest of the list. The other dimension keys, have no effect at item level.

6.2 Keys for prop environments

The keys in this section can be used in the following places:

- In the optional argument of `prop`: the key applies to that environment only, not to any other `prop` or `inlineprop` environments that may be nested within it.
- In the argument of `\proptoptions`^{P. 5}: the key will apply to every subsequent `prop` environment (in the current group), unless overridden by that environment's optional argument.
- In the optional argument of `\usepackage{propositions}`: equivalent to `\proptoptions`.
- With `\SetPropStyle` or `\DeclareNumberedStyle`, to add the key to a given style.

The keys can also be set in the optional argument of `\item`, `\ptag`, or `inlineprop`, or as part of a style used in one of those contexts, but will have no effect there, except for `labelwidth`, `labelsep`, `itemindent` and `labelindent` which will override the given dimensions for an individual `\item`.

```

topsep=<length / length list>
partopsep=<length / length list>
itemsep=<length / length list>
parsep=<length / length list>
leftmargin=<length / length list>
rightmargin=<length / length list>
labelwidth=<length / length list>
labelsep=<length / length list>
itemindent=<length / length list>
listparindent=<length / length list>

```

Override the standard L^AT_EX list dimensions. Accept the same values as `\setlength`, including rubber lengths (e.g. `itemsep=4pt plus 2pt`). Each of these keys may also be assigned a comma-separated list of dimensions: e.g. `leftmargin={2.5em, 0em}`. When the value of a key is such a list, the first dimension in the list applies at level 1, the second at level 2, etc. (Gaps in the list (e.g. `leftmargin={, 0em}`) cause the class default to be used for that level, as do levels past the end of the list.)

When used with a `prop` environment, the values of these keys can contain the special `\propformlabel` macro: see below.

`\propformlabel`

Expands to the formatted label of the (approximately) `items`-th item in this environment, computed at `\begin{prop}` *before* the list dimensions are applied, so it can be used directly in dimension expressions. Updated after each `\item` to reflect the most recently output label. Typical use:

```

% \begin{prop}[items=20,
%   leftmargin=\widthof{\propformlabel}+0.5em]
%

```

sizes the left margin to accommodate labels up to the 20th item.

`items=<n>` (initially 1)

Indicates that approximately `<n>` items are expected in this environment. The only effect of this key is in conjunction with `\propformlabel`, to compute a preview label wide enough for the `<n>`th expected item, so that dimension expressions such as `leftmargin=\widthof{\propformlabel}+0.5em` reserve enough space for the widest label. Has no effect on actual item processing or counter stepping.

`labelindent=<length / length list>` (no default)

Positions the left edge of the label box at `<length>` from the enclosing margin, adjusting `\labelwidth`, `\leftmargin`, `\labelsep`, or `\itemindent` as needed. (Since the value of any one of these dimensions can be derived from those of the other four, it never makes sense to set all five.)

tightspacing (no value)

Sets all vertical spacing to the compact defaults that the standard document classes use for level-three lists (**topsep** and **itemsep** to 2pt with stretch/shrink, **parsep** to 0pt, **partopsep** to 1pt).

nosep (no value)

Sets **\topsep**, **\itemsep**, and **\parsep** all to zero.

continue=*<boolean>* (initially **true**)

When **true**, a **prop** environment that immediately follows another **prop** (with no intervening paragraph text) replaces the normal **\topsep**-based inter-list space with **\itemsep** + **\parsep**, giving the visual appearance of a single continuous list.

6.3 Keys for **prop** and **inlineprop** environments

The keys in this section have effect in both the **prop** and **inlineprop** environments. They can also be used with **\proptoptions**, **\usepackage**, and **\SetPropStyle**, just like the keys in the previous subsection.

named style=*<style>* (initially **proposition**)

The style when an **\item** or **\ptag** has a value for **name** or **counter**, but no explicit **style**.

named ptag style=*<style>* (initially empty)

If set, overrides **named style** for **\ptag** items only.

nameless style=*<style>* (initially **numbered**)

The style used when **\item** or **\ptag** has no optional argument (no name, counter, or style).

nameless ptag style=*<style>* (initially empty)

If set, overrides **nameless style** for **\ptag** items only.

Note that the automatic assignment of a default style to an **\item** is based solely on the item's *own* optional argument. Consequently, **\begin{prop}[name=foo]\item bar** produces a *nameless*-style item with label (foo): the environment default **name=foo** is not seen during style resolution. To get the equivalent of **\item[name=foo]** (named style, bold), use **\begin{prop}[style=proposition, name=foo]\item bar**.

wrapper begin=*<code>*

wrapper end=*<code>*

Insert arbitrary *<code>* immediately before the beginning and after the end of the environment, so that the whole list can (for example) be wrapped in another environment.

Any value that contains a comma (such as a comma-separated list of options to an environment) must be enclosed in braces, so that the comma is not read as a key separator:

```
\SetPropStyle{redbox}{
  wrapper begin = {\begin{mdframed}[linecolor=red, linewidth=2pt]},
  wrapper end   = {\end{mdframed}}}
```

The wrapping environment must be one that can be split into separate begin and end code—that is, it must not read its body as a macro argument via `\collect@body` or similar.

The following command is provided for use with `wrapper begin`:

`\propoperativeleftmargin`

A read-only length giving the `leftmargin` a `prop` environment (with no optional argument) would use, given the current defaults set by `\propoptions` and the current nesting level. It is recomputed at every `\begin{prop}`, so it is valid inside a wrapper (before the environment’s own `leftmargin` has taken effect). For example, the built-in `framed` style uses this command in the value of `wrapper begin` to make the margins of framed environments match those of regular environments.

6.4 Global keys

The following key can only be set in the optional argument of `\usepackage`, or in the preamble with `\propoptions`. In the optional argument of `prop` or `inlineprop` it is ignored, with a warning. Setting it after `\begin{document}` is worse than useless: the equation hooks are installed at `\begin{document}`, so only the `nameless style=eqnum` half of the key takes effect, and `\items` start using the `equation` counter while the equations themselves keep the standard formatting.

`equations`

(no value, **global only**)

Installs hooks so that the $\text{\LaTeX}$ and `amstex` displayed equation environments (such as `equation` and `align`) use the same formatting as the special `equation` prop style. The `equation` style’s `display format` and `ref format` keys are used for generating equation numbers and cross-references to them. Redefining the `equation` style, e.g. with

```
| \SetPropStyle{equation}{format = [#1]}
```

will automatically update the hooks.

The `equations` key also sets `nameless style=eqnum`, so that top-level `\items` with no name will also use the `equation` style, while lower-level `\items` will use other kinds of numbering: for details, see section 8 below.

7 Defining and modifying styles

Styles, equivalent to bundles of key-value settings, can be defined, and used freely in the optional arguments of `\item`, `\ptag`, `prop`, `inlineprop`, and the argument of `\propoptions`. Styles can freely be combined, and the definition of one style can reference another (in which case redefining the latter style will change the effect of the former style.)

`\SetPropStyle{<name>}{<keys>}`

Defines or modifies a prop style for use with the `style` key. All item-level keys (subsection 6.1) are accepted, plus the following:

style= $\langle parent \rangle$ (no default)

Inherit from a parent style. When an item is created, the parent's settings are loaded first (recursively, if the parent itself has a parent), then this style's own keys are applied on top.

The argument may be a macro which is expanded when the item is created: for example, a style with **style**= $\{\langle style1 \rangle\}, \{\langle style2 \rangle\}, \{\langle style3 \rangle\}$ will resolve to $\{\langle style1 \rangle\}, \{\langle style2 \rangle\}, \{\langle style2 \rangle\}$ depending on the nesting depth. The built-in **numbered** and **eqnum** styles use this mechanism.

macro= $\langle command \rangle$ (no default)

A new user macro, equivalent to $\backslash item[style=\langle name \rangle]$. Any further keys given to the macro are passed to $\backslash item$.

If the style $\langle name \rangle$ already exists, $\backslash SetPropStyle$ modifies or adds keys. For example, $\backslash SetPropStyle\{proposition\}\{align=flush\}$ changes the alignment of the built-in **proposition** style while preserving its other settings.

<pre> \SetPropStyle{angle}{ labelindent = 0em, display format = \textbf{\angle\$#1\$\rangle\$}, ref format = \$\angle\$#1\$\rangle\$, macro = \angitem } \begin{prop} \angitem[Angle thesis] Everything is angular. \end{prop} No further discussion of \lastref{} is needed.</pre>	$\langle$Angle thesis$\rangle$ Everything is angular. No further discussion of $\langle$Angle thesis$\rangle$ is needed.
---	--

$\backslash DeclareNumberedStyle\{\langle name \rangle\}[\langle keys \rangle]$

Creates a new L^AT_EX counter named $\langle name \rangle$ and a matching prop style with **counter**= $\langle name \rangle$. All $\backslash SetPropStyle$ ^{P.10} keys are accepted, plus:

parent= $\langle counter \rangle$ (no default)

A parent counter; the new counter resets when the parent steps (same mechanism as $\backslash numberwithin$). A dedicated **prop** counter (stepped by each **prop** and **inlineprop**) is available for non-persistent numbering.

<pre> \DeclareNumberedStyle{P} \begin{prop} \item[counter=P] First premise. \label{p1} \item[counter=P] Second premise. \label{p2} \end{prop} From \ref{p1} and \ref{p2}\ldots</pre>	P1 First premise. P2 Second premise. From P1 and P2...
--	---

`\proplevelchoice{⟨item1, item2, ...⟩}`

Picks the entry at the current nesting depth (1-indexed, clamped to the list length). At depth 0 (outside any `prop` environment), returns the first entry. This macro is fully expandable and can be used in the `name`, `format`, and `style` keys of `\SetPropStyle` to create depth-dependent styles.

8 Built-in styles

The following prop styles are predefined. Each entry shows the defining code (using user-facing commands) and a live example. All styles can be modified with `\SetPropStyle`^{P.10}.

Text styles

`plain` Unformatted text label.

```
| \SetPropStyle{plain}{format = #1}
```

`Claim` An unformatted item.

`proposition` Bold text label. Auto-selected when `\item` has a name but no style.

```
| \SetPropStyle{proposition}{format = \textbf{#1}}
```

`Thesis` A bold-formatted item.

`thesis` Small-caps name at the text margin (flush alignment). Shortcut: `\titem`.

```
| \SetPropStyle{thesis}{format = \textsc{#1},  
  align = flush, macro = \titem}
```

`HYPOTHESIS` A flush-aligned item.

`vignette` Italic name at the text margin with a colon in the label.

```
| \SetPropStyle{vignette}{ref format = \textit{#1},  
  align = flush, label format = \textit{#1:}}
```

Example: A vignette item.

`bullet` Bullet symbol varying by depth (like `\itemize`). Shortcut: `\bitem`.

```
| \SetPropStyle{bullet}{align = center,  
  name = \proplevelchoice{\textbullet,  
    {\normalfont\textendash}, \textasteriskcentered,  
    \textperiodcentered},  
  display format = #1, macro = \bitem}
```

- A bullet item.

Numbered styles with dedicated counters

roman Roman-numeral counter, resets per section. Shortcut: `\ritem`.

```
| \DeclareNumberedStyle{roman}[parent = section,  
|   counter format = \roman{#1}, format = (#1), macro = \ritem]
```

(i) A roman-numbered item.

alph Alphabetic counter, resets per section. Shortcut: `\aitem`.

```
| \DeclareNumberedStyle{alph}[parent = section,  
|   counter format = \alph{#1}, format = (#1), macro = \aitem]
```

(a) An alphabetic item.

Level-specific styles

These styles are used by the **numbered** and **eqnum** dispatcher styles (below). They share the counters **numpropi** – **numpropv**, which reset automatically when an `\item` at the next lower level is processed.

levelone Level-1 numbered items: arabic in parentheses.

```
| \SetPropStyle{levelone}{counter = numpropi, format = (#1)}
```

leveltwo Level-2: lowercase alphabetic with dot.

```
| \SetPropStyle{leveltwo}{counter = numpropii,  
|   display format = #1., ref format = \parentref{#1}}
```

levelthree Level-3: lowercase roman in parentheses.

```
| \SetPropStyle{levelthree}{counter = numpropiii,  
|   display format = (#1), ref format = \parentref{.#1}}
```

levelfour Level-4: uppercase alphabetic with dot.

```
| \SetPropStyle{levelfour}{counter = numpropiv,  
|   display format = #1., ref format = \parentref{#1}}
```

levelfive Level-5: uppercase roman in parentheses.

```
| \SetPropStyle{levelfive}{counter = numpropv,  
|   display format = (#1), ref format = \parentref{.#1}}
```

equation Uses the **equation** counter. When the **equations** option is active, `\SetPropStyle{equation}{...}` automatically updates the `\tagform@` and `\ref` hooks for all equations.

```
| \SetPropStyle{equation}{counter = equation, format = (#1)}
```

Dispatcher styles

These styles use `\proplevelchoice` to select a level-specific style depending on the current nesting level

numbered The default nameless style. Dispatches to `levelone`–`levelfive` by nesting depth.

```
\SetPropStyle{numbered}{style = \proplevelchoice{
  levelone, leveltwo, levelthree, levelfour, levelfive}}
```

eqnum Like `numbered`, but uses `equation` at level 1. Activated by the `equations` package option (which sets `nameless style = eqnum`).

```
\SetPropStyle{eqnum}{style = \proplevelchoice{
  equation, leveltwo, levelthree, levelfour, levelfive}}
```

hierarchical Produces 1, 1.1, 1.1.1... numbering using `\parentref` and `counter` format. Defined via two helper styles:

```
\SetPropStyle{h-base}{counter = numpropi,
  counter format = \arabic{#1}, format = #1}
\SetPropStyle{h-sub}{
  counter = \proplevelchoice{numpropi, numpropii,
    numpropiii, numpropiv, numpropv},
  counter format = \arabic{#1},
  format = \parentref{.#1}}
\SetPropStyle{hierarchical}{style = \proplevelchoice{
  h-base, h-sub}}
```

Wrapper style

framed Frames a list in a `tcolorbox` (requires the `tcolorbox` package to be loaded). Change the padding with `\setlength\propframepad{...}`.

```
\SetPropStyle{framed}{
  align = flush, leftmargin = 0pt, rightmargin = 0pt,
  wrapper begin = {\begin{tcolorbox}[
    colback = white, colframe = black,
    boxrule = 0.4pt, arc = 0pt, boxsep = 0pt,
    left skip =
      \dimexpr\propoperativeleftmargin-\propframepad\relax,
    right skip =
      \dimexpr\propoperativeleftmargin-\propframepad\relax,
    left = \propframepad, right = \propframepad,
    top = \propframepad, bottom = \propframepad]},
  wrapper end = {\end{tcolorbox}},
}
```

The list runs at `leftmargin=0` inside the frame, so `\propoperativeleftmargin`^{→ P. 10} is what keeps the contents at the left margin an unframed list would have used, with the frame rule one `\propframepad` further out.

Note A framed list, with a flush label and a plain paragraph.
The unlabelled line sits flush at the same edge.

9 Cross-referencing commands

Labels placed after `\item` items (within `prop`) work with the standard `\label/\ref` mechanism. The key difference from ordinary L^AT_EX references is that `\ref` produces *formatted* output: for example, `\textbf` might be applied to the name, or the number might be wrapped in parentheses. The formatting is controlled by the `format` key (or separately by `display format` and `ref format`).

`\Ref{\label}`
`\Ref*{\label}`

Titlecase variants of `\ref` and `\ref*`: uppercases the first letter of the formatted output. (For example, if `\ref{thesis}` produces ‘the Identity Theory’, then `\Ref{thesis}` produces ‘The Identity Theory’.) The starred form suppresses the hyperlink. Note: `\Ref` only affects references produced by this package (which are stored via `\propapply`); for other references, it behaves like `\ref`.

`\nref{\label}`
`\nref*{\label}`
`\Nref{\label}`
`\Nref*{\label}`

“Naked ref.” Outputs the bare reference content with all formatting stripped. If `\ref{premise}` produces ‘(P1)’, then `\nref{premise}` produces ‘P1’. The starred form suppresses the hyperlink. `\Nref` is the titlecase variant: it uppercases the first letter of the bare content.

`\nref` can be useful in the argument of `\item`, when the the name of one proposition should depend on that of another:

<pre> \SetPropStyle{proposition}{format=(#1)} \begin{prop} \item[Phys] Everything is physical. \label{phys2} \item[\nref{phys2}*] Almost everything is physical. \label{newphys2} \item[\ref{phys2}*] This one has two sets of parentheses, which is probably not desired. \ref{newphys2} avoided this by using \nref . \end{prop} </pre>	<pre> (Phys) Everything is physical. (Phys*) Almost everything is physical. ((Phys)*) This one has two sets of parentheses, which is probably not desired. (Phys*) avoided this by using \nref. </pre>
--	--

Warning: documents where the name of one item includes a reference to that of another, and there are further references to that item, will require multiple L^AT_EX runs to resolve all references. To save time, it is better to avoid long chains of dependencies of this sort.

```
\oref[⟨prefix⟩][⟨suffix⟩]{⟨label⟩}
\oref*[⟨prefix⟩][⟨suffix⟩]{⟨label⟩}
\Oref[⟨prefix⟩][⟨suffix⟩]{⟨label⟩}
\Oref*[⟨prefix⟩][⟨suffix⟩]{⟨label⟩}
```

“Ref with options.” Extends `\ref` by injecting a prefix and/or suffix *inside* the formatting. With one optional argument, `⟨suffix⟩` is appended; with two, `⟨prefix⟩` is prepended and `⟨suffix⟩` appended. For instance, if `\ref{premise}` produces ‘(P1)’, then `\oref[*]{premise}` produces ‘(P1*)’ and `\oref[old~]{*}{premise}` produces ‘(old~P1*)’.

`\Oref` is the titlecase variant; the starred forms suppress the hyperlink.

`\oref` can also be useful in the name of `\items`, if one wants the display format for the modified item to depend on that originally used

<pre>\begin{prop} \label{premise} \item[style=plain,name=\oref[*]{phys2}] This will use boldface and parentheses because the original referenced item did. \end{prop}</pre>	(Phys*) This will use boldface and parentheses because the original referenced item did.
---	---

Another handy use for `\oref` is in combination with `\nref` to refer to ranges:

<pre>The first two numbered examples in this document were were \oref[--\nref{atomism}]{atoms}.</pre>	The first two numbered examples in this document were were (1–2).
---	--

```
\lastref[⟨prefix⟩]{⟨suffix⟩}
\Lastref[⟨prefix⟩]{⟨suffix⟩}
```

Produces a reference (with no hyperlink) to the most recently processed `\item` or `\ptag`, even without a `\label`. Useful for back-references in running text. With one argument, `⟨suffix⟩` is appended; with two, `⟨prefix⟩` is also prepended. Use `\lastref{}` for a plain reference. `\Lastref` is the titlecase variant.

```
\nlastref
\Nlastref
```

Like `\lastref{}`, but returns the bare content without formatting. `\Nlastref` is the titlecase variant.

```
\parentref[⟨prefix⟩]{⟨suffix⟩}
\Parentref[⟨prefix⟩]{⟨suffix⟩}
```

Inside a nested `prop` (or `inlineprop`), produces a formatted reference to the most recent item of the enclosing level. Same argument convention as `\lastref`. `\Parentref` is the titlecase variant.

`\nparentref`
`\nParentref`

Like `\parentref{}` but returns the bare content without formatting. Takes no arguments; simply output any desired suffix directly afterwards. `\nParentref` is the titlecase variant.

`\parentref`^{→P.16} and `\nparentref` are useful for making subitems whose names derive from their parent's:

```
\DeclareNumberedStyle{inner}[
  counter format=\alph{inner},
  display format=#1.]
\begin{prop}
  \item[OI] Outer item.
  \label{outer2}
  \begin{prop}
    \item[style=inner,
      format=\parentref{.#1}]
    \label{dsub1}
    Ref: \ref{dsub1},
    naked: \nref{dsub1}.
    \item[style=inner,
      display format=#1.,
      ref format=\parentref{.#1}]
    \label{dsub2}
    Ref: \ref{dsub2},
    naked: \nref{dsub2}.
  \end{prop}
\end{prop}
```

OI Outer item.

OI.a Ref: **OI.a**, naked: a.

b. Ref: **OI.b**, naked: b.

The built-in `leveltwo` and `levelthree` styles have `ref format=\parentref{#1}` and `ref format=\parentref{.#1}`, respectively, so that if the parent references as ‘(P1)’, a `leveltwo` sub-item references as ‘(P1a)’ and `\nref` returns just ‘a’.

9.1 How cross-referencing works: `\propapply`

`\propapply{<template>}{<content>}`

Internally, each reference is stored in the `.aux` file as `\propapply{<template>}{<content>}`.

The `<template>` contains formatting with the placeholder `\propfmtarg` where content appears. At reference time, `\propapply` evaluates the template with `\propfmtarg` bound to `<content>`. The `\oref`^{→P.16} and `\nref`^{→P.15} commands work by locally redefining `\propapply`.

In normal use, you need not interact with `\propapply` directly.

`\propfmtarg`

Placeholder used inside templates; expands to the content argument of the enclosing `\propapply`.

10 Compatibility

The propositions package is designed to work with `hyperref`, `cleveref`, `amsmath`, and `tcolorbox`.

- `amsmath` is required for `\ptag` and the `equations` option.
- With `hyperref`, all cross-referencing commands generate hyperlinks, as usual.
- With `cleveref` loaded, all proposition items are assigned to a private `prop` “reference type”, which by default has no special name, so `\cref` and `\ref` will generate the same output. The `crefname` key can override this.
- The built-in `framed` style requires `tcolorbox` to be loaded.

None of these packages has to be loaded in any particular order relative to `propositions`: each is detected either at load time or at `\begin{document}`, whichever is needed. The usual external conventions still apply—`hyperref` late, and `cleveref` after `hyperref`—which gives:

```
\usepackage{amsmath}      % if using \ptag
\usepackage{tcolorbox}    % if using the framed style
\usepackage{hyperref}
\usepackage{propositions}
\usepackage{cleveref}     % if used
```

11 Known issues

When using `\ptag` with a named counter (e.g. `\ptag[counter=P]`) inside an `amsmath` equation environment, `hyperref` may emit warnings of the form:

```
pdfTeX warning: destination with the same
identifier (name{equation.N}) has been already
used, duplicate ignored
```

These warnings are harmless and do not affect the correctness of cross-references.